

CSE-2340 - Week 1, Session C3

SEMANTIC HTML - STRUCTURE, MEANING, LINKS, IMAGES AND FORMS

Big Question

How do I turn a blank HTML file into a page with clear structure, meaningful content, usable links, accessible images, and a basic form?

1. Learning Objectives

- Explain HTML and distinguish it from CSS and JavaScript.
- Distinguish an element, tag, attribute, and document structure.
- Build a logically organized page using semantic HTML.
- Use h1–h6 appropriately and create meaningful text structure.
- Create links, images and basic forms correctly.
- Use href, src, alt, id, name and common form attributes appropriately.
- Explain why semantic HTML supports accessibility, maintainability and clearer document structure.
- Inspect page structure using browser DevTools.
- Improve poorly structured HTML without depending on CSS.

2. Target Answer

By the end of C3, you should be able to open the project created in C2 and build a small semantic page without needing CSS. It should contain a meaningful document structure, readable content, useful links, an accessible image and a basic contact form. You should be able to explain why each major element belongs where it is.

3. What Is HTML?

HTML stands for HyperText Markup Language. It is the markup language used to describe the structure and meaning of content on a web page.

Technology	Main responsibility	Mental model
HTML	Structure and meaning	What is this content?
CSS	Presentation and layout	How should it look?
JavaScript	Behavior and interaction	What should happen?

Remember

HTML gives structure and meaning. CSS will later control presentation. JavaScript will later control behavior.

4. Element, Tag and Attribute

4.1 Element

An element is the complete HTML construct, including its opening tag, content and closing tag when applicable.

```
<p>Hello, students!</p>
```

4.2 Tag

A tag is the markup syntax used to start or identify an element. In the paragraph example, `<p>` is the opening tag and `</p>` is the closing tag.

4.3 Attribute

An attribute provides additional information about an element and is normally written inside its opening tag.

```
<a href="https://example.com">Visit Example</a>
```

Concept	Example	Meaning
Element	<code><p>Hello</p></code>	Complete paragraph
Opening tag	<code><p></code>	Starts the paragraph
Closing tag	<code></p></code>	Ends the paragraph
Attribute	<code>href="..."</code>	Additional information
Attribute value	<code>"https://example.com"</code>	Value assigned to attribute

5. The Basic HTML Document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
</head>
<body>
  <h1>My Student Profile</h1>
  <p>I am learning frontend development.</p>
</body>
</html>
```

5.1 <!DOCTYPE html>

The doctype tells the browser to interpret the document using modern HTML standards.

5.2 html

The html element is the root of the document. `lang="en"` declares the primary language.

5.3 head

head contains document information such as title, metadata, stylesheets and scripts.

5.4 body

body contains the visible and interactive page content.

6. Semantic HTML: Meaning Before Appearance

Semantic HTML means choosing elements according to what the content means, not merely how you want it to look.

Element	Meaning / intended use
header	Introductory content for a page or section
nav	A major group of navigation links
main	Primary content of the document
section	Thematic group of related content
article	Self-contained content that can stand independently
footer	Closing information for a page or section
h1–h6	Heading hierarchy
p	Paragraph
a	Hyperlink
img	Image

Important

Semantic does not mean 'looks better'. It tells browsers, assistive technologies, developers and other tools what the content represents.

7. Headings and Document Hierarchy

HTML provides six heading levels: h1 through h6. They represent a hierarchy of topics.

```
<h1>Web Development Course</h1>
<h2>HTML</h2>
<h3>Semantic HTML</h3>
<h3>HTML Forms</h3>
<h2>CSS</h2>
<h3>Responsive Design</h3>
```

- Use h1 for the main page topic.
- Use h2 for major sections under the main topic.
- Use h3 for subsections under an h2.
- Do not choose heading levels only because their default text looks larger or smaller.
- Heading hierarchy communicates document structure.

8. Text Content

8.1 Paragraphs

```
<p>Semantic HTML makes web pages easier to understand and maintain.</p>
```

8.2 Strong and Emphasis

```
<p><strong>Important:</strong> Submit before the deadline.</p>
<p><em>Note:</em> This page is for practice.</p>
```

Use strong and em when the meaning calls for importance or emphasis. Do not use them simply as replacements for CSS styling.

8.3 Lists

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<ol>
  <li>Open VS Code</li>
  <li>Open the project</li>
  <li>Start Live Server</li>
</ol>
```

ul creates an unordered list; ol creates an ordered list; each item uses li.

9. Links: Connecting the Web

The anchor element creates hyperlinks.

```
<a href="https://developer.mozilla.org/">Visit MDN</a>
<a href="about.html">About Us</a>
<a href="#contact">Go to Contact</a>
```

9.1 The href Attribute

Link type	Example	Use
External URL	href="https://example.com"	Another website
Relative page	href="about.html"	Another page in the project
Section link	href="#contact"	Location on same page
Email	href="mailto:student@example.com"	Email action

Use meaningful link text. 'Read the HTML documentation' communicates more than a link that only says 'click here'.

10. Images

```

```

- src tells the browser where the image file is located.
- alt provides a text alternative describing the image's purpose.
- img is a void element and does not use a closing tag.
- Use meaningful alt text when the image communicates information.

- For purely decorative images, an intentional empty alt value can be used: alt="".

Accessibility rule

Do not simply omit alt. Make an intentional decision about the image's alternative text.

11. File Paths

```
project/
├── index.html
├── about.html
├── images/
│   └── profile.jpg
├── css/
│   └── style.css
```

From index.html	Meaning
about.html	File in same folder
images/profile.jpg	File inside images
css/style.css	File inside css
../about.html	Go up one directory

12. Forms: Collecting User Input

```
<form>
  <label for="name">Name</label>
  <input id="name" name="name" type="text">

  <label for="email">Email</label>
  <input id="email" name="email" type="email">

  <label for="message">Message</label>
  <textarea id="message" name="message"></textarea>

  <button type="submit">Send Message</button>
</form>
```

12.1 form

Groups related controls used to collect or submit user input.

12.2 label

Identifies a form control. Connecting for to the control's id improves usability and accessibility.

12.3 input

Provides many control types including text, email, password, number, date, checkbox and radio.

12.4 textarea

Provides multi-line text input.

12.5 select and option

```
<label for="department">Department</label>
<select id="department" name="department">
  <option value="cse">CSE</option>
  <option value="eee">EEE</option>
</select>
```

12.6 button

Creates a clickable button. type="submit" makes it submit the form.

13. The label–id Connection

```
<label for="student-name">Student Name</label>
<input id="student-name" name="studentName" type="text">
```

- for identifies the control the label describes.
- id identifies the control in the document.
- name identifies the field when form data is submitted.
- id and name can have the same value, but they serve different purposes.

14. Complete Worked Example — Student Profile

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
</head>
<body>
  <header>
    <h1>Student Profile</h1>
    <nav>
      <a href="#about">About</a>
      <a href="#skills">Skills</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About Me</h2>
      
      <p>I am learning modern frontend development.</p>
```

```
</section>

<section id="skills">
  <h2>Skills</h2>
  <ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
  </ul>
</section>

<section id="contact">
  <h2>Contact</h2>
  <form>
    <label for="name">Name</label>
    <input id="name" name="name" type="text">

    <label for="email">Email</label>
    <input id="email" name="email" type="email">

    <label for="message">Message</label>
    <textarea id="message" name="message"></textarea>

    <button type="submit">Send</button>
  </form>
</section>
</main>

<footer>
  <p>© 2026 Student Profile</p>
</footer>
</body>
</html>
```

15. Read the Structure, Not the Appearance

The page may look plain. That is expected. Do not use CSS to make bad structure look good. First make the HTML meaningful; later CSS will control spacing, colors, typography, layout and responsiveness.

Question	HTML answer
Main topic	h1
Major areas	section + h2
Navigation	nav
Primary content	main
Independent content	article when appropriate
Collected information	form + controls
Link destination	a + href
Image	img + src + alt
Closing information	footer

16. Nesting: Parent and Child Elements

```
<main>
  <section>
    <h2>Skills</h2>
    <p>I am learning HTML.</p>
  </section>
</main>
```

main contains section. The section contains h2 and p. This parent–child structure becomes part of the browser's DOM.

17. Common Beginner Mistakes

Mistake	Why it is a problem	Better approach
Using div for everything	Meaningful structure is lost	Use semantic elements when their meaning fits
Choosing headings by size	Hierarchy becomes misleading	Choose heading levels by structure
Missing alt	Accessibility suffers	Make an intentional alt decision
Vague link text	Destination is unclear	Use meaningful link text
Missing form labels	Controls may be unclear	Connect label and control
Broken relative path	Resource cannot load	Check file location and path
Using br for layout	HTML becomes responsible for presentation	Use CSS for layout
Visible content in head	Incorrect document structure	Put visible content in body

18. Live / Guided Practice

Create student-profile.html inside the Week 1 project created in C2.

1. Add the standard HTML document structure.
2. Set the page title to Student Profile.
3. Create a header containing an h1 and nav.
4. Add three navigation links: About, Skills and Contact.
5. Create a main element with three sections.
6. Add an About section with an h2, image and paragraph.
7. Add a Skills section with an h2 and unordered list.
8. Add a Contact section with an h2 and form.
9. Add labels and controls for name, email and message.
10. Add a submit button and footer.
11. Run the page with Live Server.
12. Open DevTools and inspect the DOM tree.
13. Change one piece of content and verify the browser updates.

19. Debugging Challenge

Improve this deliberately poor page without adding CSS:

```
<div>
  <div>My Portfolio</div>
  <div><a href="about.html">Click here</a></div>
```



```
<div>
  
  <div>About me</div>
  <div>I am a CSE student.</div>
</div>
<div>
  <input type="text">
  <button>Send</button>
</div>
</div>
```

- Which element should represent the page header?
- Which element should contain navigation?
- Which element should contain the main content?
- Which heading level should be used?
- What should the image alt text be?
- Where should a label be added?
- What element should group the input and button?

20. DevTools Practice

1. Open your student-profile page.
2. Open Developer Tools and select Elements/Inspector.
3. Expand html, body, main and each section.
4. Identify parent and child relationships.
5. Temporarily change text in DevTools.
6. Refresh and observe that the temporary change disappears.
7. Return to VS Code and make the real change in source HTML.

Important distinction

Your HTML file is source code. The DOM shown in DevTools is the browser's current representation of the document. JavaScript can later modify the DOM.

21. Mini Exercise - Course Information Page

Build a CSE-2340 course information page without copying the worked example.

- One h1 for the course name.
- Navigation links to Overview, Topics and Contact.
- An Overview section.
- A Topics section with an unordered list.
- An image with meaningful alt text.
- A Contact section with name, email and message controls.
- A submit button and footer.

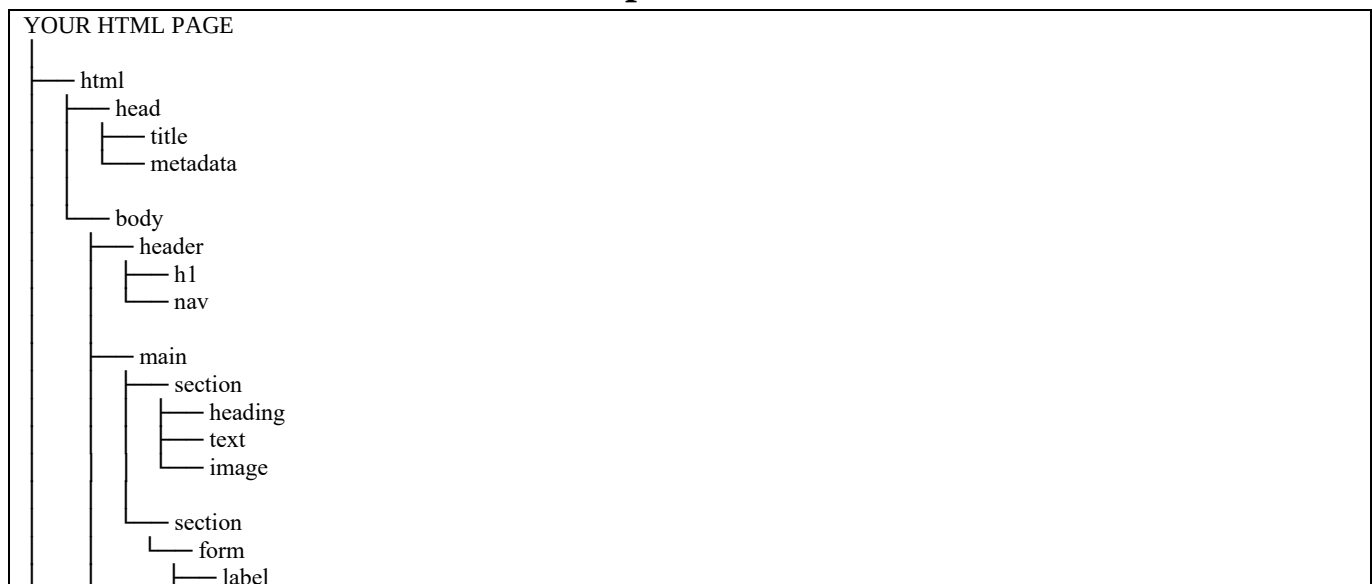
22. Check for Understanding

Question	What a strong answer should mention
Why semantic HTML?	Meaning, accessibility, maintainability and structure
Why not choose h3 because it looks smaller?	Heading level represents hierarchy; CSS controls appearance
What does href do?	Specifies a link destination
What does src do?	Specifies a resource location
Why alt?	Provides an alternative text description for an image
Why label + for/id?	Clearly associates a label with a control
section vs article?	Section groups related content; article is self-contained
Why not use br for spacing?	Layout belongs to CSS

23. Essential Glossary

Term	Student-friendly definition
HTML	Markup language used to structure web documents.
Markup	Syntax used to describe structure or meaning.
Element	A complete HTML construct.
Tag	Markup syntax used to start or identify an element.
Attribute	Additional information attached to an element.
Semantic HTML	HTML that communicates the meaning or role of content.
Heading	A title for a page or section, using h1–h6.
Hyperlink	A connection to another page, resource or location.
Relative path	A path interpreted relative to the current document.
alt text	Text alternative describing an image's meaningful purpose.
Form	A group of controls for collecting or submitting input.
Label	Text identifying a form control.
Input	A form control for collecting a value.
Textarea	A multi-line text input control.
Select	A control for choosing from options.
DOM	The browser's in-memory representation of the document.
Accessibility	Designing interfaces usable by people with different abilities and technologies.

24. The One Mental Model to Keep





If you remember only one thing

Write HTML for meaning and structure first. Do not ask, 'Which tag makes this look bigger?' Ask, 'What is this content?'

25. Final Self-Check

- I can explain what HTML is used for.
- I can distinguish HTML from CSS and JavaScript.
- I can explain element, tag and attribute.
- I can create a correct basic HTML document.
- I can use header, nav, main, section, article and footer appropriately.
- I can create a logical heading hierarchy.
- I can create paragraphs, lists and meaningful links.
- I can use src and alt correctly for images.
- I can build a basic form and connect labels to controls.
- I understand id and name.
- I can inspect the DOM in DevTools.
- I can identify poor HTML structure and improve it.
- I can build a small semantic page without CSS.

26. Homework

14. Create a personal profile page using only HTML.
15. Use header, nav, main, at least three sections and footer.
16. Use one h1 and a logical h2 hierarchy.
17. Include one image with meaningful alt text.
18. Include an unordered list of skills or interests.
19. Include three meaningful links.
20. Include a contact form with name, email, message and submit button.
21. Use at least one local resource with a relative path.
22. Run the page with Live Server.
23. Inspect the DOM and verify the hierarchy.
24. Keep the project ready for the Git work in C4.

28. Connection to C4 — Git Basics

Your semantic HTML page becomes the first meaningful project artifact for this week. In C4 you will initialize or continue a Git repository, create meaningful commits and push the project to GitHub. Keep the project organized so that version control records real progress.

- C3 creates the semantic HTML project.
- C4 records the project using version control.
- Later mini projects will build on the same workflow.
- Git is part of the development workflow, not a separate subject.

29. Further Learning Direction

C3 gives you the structural foundation of a web page. Next, you will extend HTML with practical navigation, forms and tables, then move into accessibility and CSS. Later CSS will control visual presentation and JavaScript will add behavior. The semantic structure you create now will remain underneath the interfaces you eventually build with React.